Digital Portfolio Website (Portfoliofy): Project Documentation

This document describes the repository `Digital_Portfolio_Website` (internal project name `DigitalPortfolioBackend`, product name shown in the UI and PDF output: "Portfoliofy"). It is based on a direct reading of the source code. Where the repository's own prose documents (README, guides) disagree with the code, the code is treated as the source of truth and the discrepancy is noted.

1. Overview and purpose

Digital Portfolio Website is a full-stack web application that lets people register, build an online professional portfolio, and share it with others. A portfolio consists of a title, description, profile image, contact details, and five optional sections: projects, education, work experience, skills and social media links. The owner can mark the whole portfolio public or private and can independently show or hide each of the five sections.

The main capabilities found in the code are:

- Registration, login and logout using ASP.NET Core cookie authentication, with BCrypt password hashing.
- Create, read, update and delete (CRUD) of portfolios and each child section.
- Image upload for profile pictures and project images (server-side resize and re-encode to JPEG).
- A private-portfolio access workflow: a signed-in user requests access, the owner is notified in-app and by email, and an approved request unlocks the private portfolio.
- An in-app notification center and portfolio view logging.
- An admin role (users and portfolios listing, analytics, delete portfolio, password reset), promoted via a secret PIN.
- Portfolio export as a self-contained styled HTML document (the endpoint is named `/pdf` but returns HTML).
- An OpenAI-backed chatbot used for navigation and for portfolio advice, project descriptions and skill suggestions.
- Public statistics endpoints used by the landing page.

The repository contains both the ASP.NET Core API (at the repository root) and a React single-page application (in `Frontend/`). The README describes only the backend in detail, and several features it lists as "Future Enhancements" (view analytics, AI content suggestions) are partly implemented in code.

2. Tech stack

Layer	Technology	Version or note
Backend runtime	.NET / ASP.NET Core Web API	`net9.0`, nullable enabled
ORM	Entity Framework Core + Pomelo MySQL provider	EF Core 9.0.0, Pomelo 9.0.0
Database	MySQL	database name `dpw_db` in config
Auth	Cookie authentication	cookie `DigitalPortfolioAuth`, 24 h sliding
Password hashing	BCrypt.Net-Next	4.0.3
API docs	Swashbuckle (Swagger)	8.1.1, Development only
Email	`System.Net.Mail.SmtpClient`	Gmail SMTP defaults
AI	OpenAI Chat Completions over `HttpClient`	model `gpt-3.5-turbo`
Image processing	SixLabors.ImageSharp	used in code, see gap in section 8
Frontend	React 18, React Router 6, Vite 7	JavaScript (JSX), ES modules
Styling	Tailwind CSS 3.4 plus hand-written CSS	PostCSS and Autoprefixer
HTTP client	axios 1.6	`withCredentials: true`
Client export	html2pdf.js 0.12	used on the portfolio view page
Visual effects	Three.js r134 and Vanta.js NET	loaded from CDN in `index.html`

Layer	Technology	Version or note
CI	GitHub Actions	four workflow files

Packages referenced in `DigitalPortfolioBackend.csproj` but not used by any source file read: `DinkToPdf`, `IronPdf`, `Microsoft.AspNetCore.Authentication.JwtBearer`, `Microsoft.IdentityModel.Tokens`, `Microsoft.AspNetCore.Mvc` 2.3.0.

3. Repository structure

```
Digital_Portfolio_Website/
|-- Program.cs                      # app bootstrap, DI, CORS, auth, middleware pipeline
|-- DigitalPortfolioBackend.csproj  # net9.0 project, NuGet packages
|-- DigitalPortfolioBackend.sln
|-- appsettings.json                # template config with placeholders
|-- appsettings.Production.json     # identical placeholder template
|-- Properties/launchSettings.json  # http://localhost:5163, https://localhost:7064
|-- Controllers/                    # 12 API controllers
|-- Models/                         # EF Core entities
|-- DTOs/                           # request/response shapes
|-- Data/AppDbContext.cs            # DbSets and relationship config
|-- Migrations/                     # 2 migrations plus model snapshot
|-- Middleware/AuthMiddleware.cs    # console logging of request identity
|-- Services/                       # email, OpenAI, HTML export generators
|-- Frontend/                       # React + Vite single-page app
|   |-- index.html, vite.config.js, tailwind.config.js, postcss.config.js, eslint.config.js
|   |-- services/api.js             # axios instance and API wrappers
|   `-- src/ (main.jsx, App.jsx, register.jsx, pages/, components/, contexts/, data/, *.css)
|-- .github/workflows/              # ci-cd.yml, deploy.yml, github-pages.yml, main.yml
|-- *.bat, setup.ps1               # Windows start and setup helpers
|-- package.json                    # root, only Tailwind dev dependencies
|-- *.md                            # README, guides, prompt files
`-- images, pdf, html               # diagrams and sample export
```

Folder and file roles

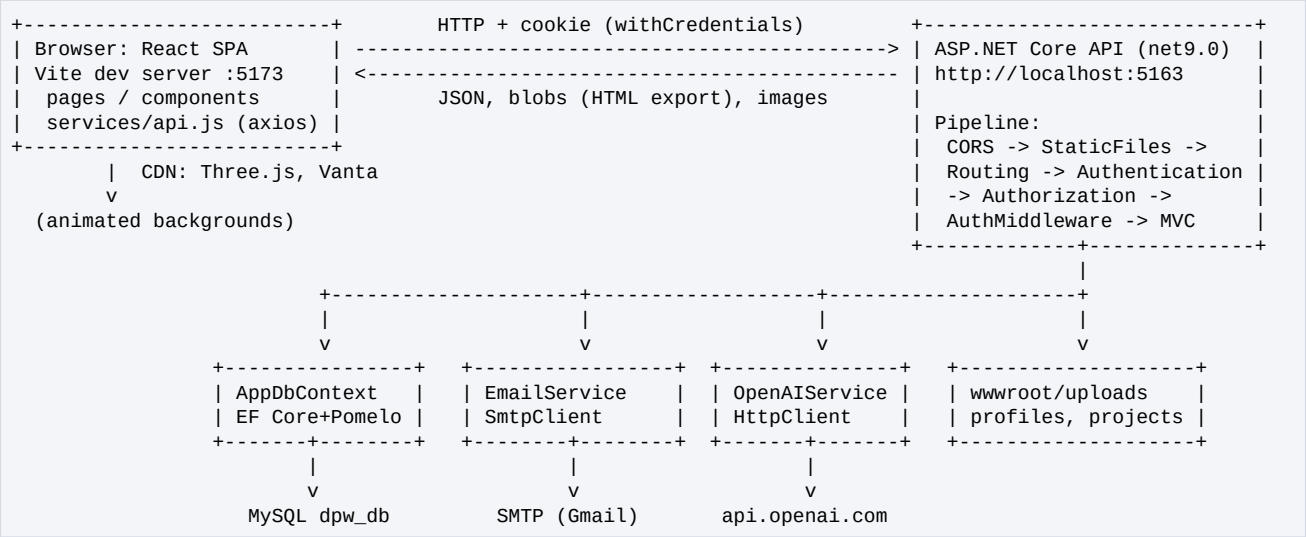
Path	Role
`Program.cs`	Registers controllers, Swagger, MySQL DbContext, PDF/Email/OpenAI services, CORS, cookie auth; builds the pipeline
`Controllers/`	Auth, Admin, Portfolio, Project, Education, Experience, Skill, AccessRequest, Notification, Chatbot, ImageUpload, Statistics
`Models/`	`User`, `Portfolio`, `Project`, `Education`, `Experience`, `Skill`, `SocialMediaLink`, `AccessRequest`, `Notification`, `PortfolioViewLog`, plus `PortfolioCreationState` (not mapped to the DB)
`DTOs/`	Create/update/response objects such as `PortfolioDto`, `CreatePortfolioDto`, `UserRegisterDto`, `UserLoginDto`, `UpdateProfileDto`, `ForgotPasswordDto`, `AccessRequestDto`
`Services/EmailService.cs`	SMTP sending and HTML email templates
`Services/OpenAIService.cs`	Calls OpenAI chat completions with canned prompts
`Services/ProfessionalPDFGenerator.cs`	699-line builder of a styled HTML document from a portfolio
`Services/PDFGeneratorService.cs`	Thin wrapper over the generator with logging
`Frontend/services/api.js`	Single axios instance and per-resource API objects
`Frontend/src/pages/`	Route-level screens
`Frontend/src/components/`	Navigation, Chatbot widget, header/footer, loading, splash, create/view widgets
`Frontend/src/contexts/PortfolioContext.jsx`	In-memory portfolio state used by the legacy demo pages
`Frontend/src/data/examplePortfolios.js`	Sample portfolio data
`Portfoliofy-Portfolio-2025-09-24.html`	Sample exported portfolio
`Project proposal.pdf`, `er diagram.png`, `system_architecture (1).png`, `workflow.png`, `workflow.webp`	Design artifacts (binary, not analysed)

Path	Role
`ARCHITECTURE_DIAGRAMS.md`, `DEPLOYMENT_GUIDE.md`, `CONFIG_SETUP.md`, `QUICK_START_README.md`, `ACCESS_REQUEST_*.md`	Extra prose docs
`MEGA_PROMPT.md`, `AI_WORKFLOW_PROMPT.md`, `PROFESSIONAL_DOCUMENTATION_PROMPT.md`	AI prompt files; the last is empty
`quick-start.bat`, `start-*.bat`, `setup.ps1`	Developer launch and environment-check scripts

Notable assets: `Frontend/src/assets/` holds `Jade-logo.svg`, `portfolio-logo.svg`, `portfolio-alt-logo.svg`, `react.svg` and `digital portfolio logo.png`. `Frontend/public/` holds only `vite.svg`. There is no `wwwroot/` directory committed, although the upload code and `.gitignore` expect `wwwroot/uploads/{profiles,projects}`.

4. Architecture

The system is a classic two-tier web application: a browser SPA talking JSON over HTTP to a monolithic ASP.NET Core API that persists to MySQL. The API also calls two external services (SMTP and OpenAI) and serves uploaded images as static files.



Components

- **React SPA.** Route-level pages call the shared axios instance. Authentication state is not held client-side; the browser simply sends the HttpOnly cookie and the SPA asks `/Auth/check-session` (cached 30 seconds in memory) to learn whether it is signed in.
- **API controllers.** Thin controllers that query `AppDbContext` directly; there is no repository or service layer for business logic apart from email, OpenAI and export generation.
- **Persistence.** Ten mapped entities, all relationships cascade-delete except `PortfolioViewLog.Viewer`, which is set to null.
- **Static files.** Uploaded images are served from `wwwroot`. A custom `OnPrepareResponse` handler echoes the request `Origin` back in `Access-Control-Allow-Origin` with credentials enabled.

Control and data flow

1. **Login.** SPA posts credentials to `/api/Auth/login`. The controller verifies the BCrypt hash, builds claims (NameIdentifier, Email, FullName, Role), and calls `SignInAsync`. The browser stores the `DigitalPortfolioAuth` cookie.
2. **Browsing.** `Dashboard` and `PublicPortfolios` call `/Portfolio/all-visible`. Each portfolio DTO includes only the sections whose visibility flag is true (owner sees all).
3. **Opening a portfolio.** `GET /Portfolio/{id}` checks ownership; if the portfolio is private and the caller is not the owner it requires an `Approved` row in `AccessRequests`, otherwise returns 403. A `PortfolioViewLog` row is

written, and a `PortfolioViewed` notification goes to the owner when a signed-in non-owner views.

4. **Access request.** `POST /AccessRequest/portfolio/{id}` creates an `AccessRequest` (Pending), a `Notification` for the owner, and sends an email to the owner.
5. **Editing.** `PortfolioEdit` loads the portfolio and each section through separate endpoints and writes changes with per-section calls; ownership is verified in the child controllers.
6. **Export.** `GET /Portfolio/{id}/pdf` builds an HTML document in memory and returns it as `text/html` with an `.html` filename.
7. **Chatbot.** The widget posts conversation messages to `/Chatbot/ask-with-options`; the service forces a JSON-formatted reply that can carry menu options and a redirect action the widget executes with React Router.

5. Detailed walkthrough

5.1 Backend bootstrap: `Program.cs`

- Registers controllers, Swagger, and `AppDbContext` using `UseMySql` with `ServerVersion.AutoDetect` on the `DefaultConnection` string (this opens a DB connection at startup).
- Registers `ProfessionalPDFGenerator`, `PDFGeneratorService` and `EmailService` as scoped; `SmtpSettings` as a singleton built from the `Smtp` section; `OpenAIService` through `AddHttpClient`.
- CORS policy `AllowFrontend` reads the `AllowedOrigins` string array and defaults to `localhost:3000`, `5173` and `5174`; it allows any header and method with credentials.
- Cookie authentication: name `DigitalPortfolioAuth`, `HttpOnly`, 24 h sliding expiry, `SameSite=Lax`, `SecurePolicy=None`. Login and access-denied redirects are replaced with plain 401 and 403 responses.
- Pipeline order: Swagger (Development) or HTTPS redirect (otherwise), CORS, StaticFiles, Routing, Authentication, Authorization, `AuthMiddleware`, controllers.

5.2 Middleware

`AuthMiddleware` writes one console line per request stating whether the user is authenticated (with user id) and the path. It makes no decisions.

5.3 Controllers and endpoints

The route template is `api/[controller]`. "Auth" means `[Authorize]`; "Public" means no attribute.

AuthController

Endpoint	Access	Behaviour
`POST forgot-password`	Public	Sends the admin contact email to the given address in a background task; returns admin name, email, phone and a `sent` flag that equals whether the user exists
`POST register`	Public	Validates fields, rejects duplicate email, hashes password, saves user, signs in, sends a welcome email in the background
`POST login`	Public	Verifies BCrypt hash, signs in, returns `{ message }` only
`POST logout`	Public	Signs out
`POST promote?email=`	Public + PIN	Header `X-Admin-Pin` must equal `Admin:Pin`; sets `Role = "Admin"`
`GET check-session`	Cookie	Returns `isAuthenticated`, `userId`, `email`, `role` or 401
`PUT profile`	Cookie claim	Updates name, email and optionally password (current password required)

PortfolioController (about 1100 lines)

Endpoint	Access	Behaviour
`GET public`	Public	All portfolios with sections filtered by visibility flags
`GET {id}`	Public, gated	Owner or public or approved requester; logs a view and may create a notification

Endpoint	Access	Behaviour
`GET my-portfolios`	Auth	Caller's portfolios
`GET accessible`	Auth	Owned plus approved-access portfolios
`GET all-visible`	Public	Listing for browse pages, flag-filtered sections
`POST`	Auth	Creates a portfolio from `CreatePortfolioDto`
`PUT {id}`	Auth	Owner-only update
`DELETE {id}`	Auth	Owner-only delete (403 otherwise)
`GET {id}/pdf`	Public	HTML export built by `PDFGeneratorService`
`GET/POST/PUT/DELETE {portfolioId}/social-media[/{id}]`	Reads public, writes Auth	Social link CRUD

A private helper `GetCurrentUserId` tries several claim types and prints extensive claim and Authorization-header diagnostics to the console on failure.

Project, Skill, Education, Experience controllers

All follow one pattern: `GET portfolio/{portfolioId}` (private portfolios restricted to the owner), `GET {id}`, authenticated `POST portfolio/{portfolioId}`, `PUT {id}`, `DELETE {id}`, each verifying the portfolio belongs to the caller. Education and Experience also expose an unfiltered `GET` (list everything) and Education has an extra `POST` without a portfolio id. Dates are exchanged as `yyyy-MM-dd` strings; a missing end date becomes an empty string. Skill `Level` is an integer 1 to 5.

AccessRequestController (class-level `[Authorize]`)

- `POST portfolio/{portfolioId}`: body `AccessRequestDto { message }`. Rejects a second request from the same user for the same portfolio, creates the request, a notification and an email (awaited inline).
- `GET debug/{portfolioId}`: marked `[AllowAnonymous]` ("temporarily"); returns owner id and name, privacy state, the caller's requests and the owner's five latest notifications.

NotificationController (`Authorize`): `GET debug`, `GET test-db`, `GET my` (latest 100 for the caller), `POST {id}/read`, `POST read-all`.

AdminController (`[Authorize(Roles = "Admin")]`)

Endpoint	Behaviour
`GET users`	Id, name, email, role
`GET portfolios`	All portfolios with all sections as DTOs
`GET portfolios/{id}/full`	Portfolio and owner, entities returned directly
`GET portfolios/{id}/analytics`	Projects by year, skills by level, experience months
`DELETE portfolios/{id}`	Hard delete
`POST users/{id}/reset-password`	Requires `X-Admin-Pin`; generates a temporary password and returns it in the response

ChatbotController (`Authorize`): `POST message`, `POST ask-with-options`, `POST analyze-portfolio`, `POST generate-project-description` (name and technologies required), `POST suggest-skills`, `POST review-portfolio`, and `GET history` (always returns an empty list).

ImageUploadController: `POST profile` (max 800 px) and `POST project` (max 1200x800); both `[Authorize]`. Only extension is checked (jpg, jpeg, png, gif). The image is loaded with ImageSharp, optionally resized, and always saved as JPEG at quality 75 under a GUID filename, even for `.png` and `.gif` input. The response is `{ url: "/uploads/<kind>/<guid>.<ext>" }`.

StatisticsController (public): `GET` returns user and portfolio counts, active users, portfolios in the last month, formatted counts, and hard-coded `uptime = "99.9%"` and `support = "24/7"`. `GET dashboard` returns totals plus recent users (name and email) and recent portfolios.

5.4 Services

- **EmailService.** `SendEmailAsync` skips silently when `Smtp:Host` is empty, otherwise sends HTML mail with `SmtpClient`. Templates: registration confirmation, access requested, access decision (approved or rejected), forgot-password admin contact, and the access-request notification used by the controller.
- **OpenAIService.** Adds a `Bearer` header from `OpenAI:ApiKey` to the injected client. All calls go to `https://api.openai.com/v1/chat/completions` with `gpt-3.5-turbo`. `GetChatResponseAsync` uses a fixed "portfolio assistant" system prompt (max 500 tokens, temperature 0.7). Helper methods wrap analysis, project description, skill suggestion and section review prompts. `GetChatResponseWithOptionsAsync` uses a navigation-bot system prompt that demands JSON `{response, options, action}` and falls back to plain text when parsing fails.
- **ProfessionalPDFGenerator and PDFGeneratorService.** Load a portfolio with all children and produce a full HTML page with embedded CSS. Image URLs that are relative are prefixed with `http://localhost:5163`; a missing profile image maps to a `via.placeholder.com` URL. On any error the generator returns an error HTML page rather than throwing.

5.5 Frontend

Entry and routing. `main.jsx` mounts `App` and overrides `console.log`, `console.error` and `console.warn` to hide React DevTools messages and several API error strings. `App.jsx` wraps everything in `PortfolioProvider` and `BrowserRouter` and defines these routes:

Route	Component
`/`, `/live`	`LiveWallpaperWelcome` (landing with animated background and statistics)
`/welcome`	`WelcomePage`
`/login`, `/register`	`Login`, `Register` (`src/register.jsx`)
`/dashboard`, `/browse`, `/portfolios`	`Dashboard`
`/portfolio/create`	`PortfolioCreate`
`/portfolio/edit/:id`	`PortfolioEdit`
`/portfolio/view/:id`	`PortfolioView`
`/portfolio/manager`	`PortfolioManager`
`/profile`	`Profile`
`/edit`, `/view`, `/showcase`	`EditPage`, `ViewPage`, `ShowcasePage` (legacy, context-based)
`/admin`	`AdminDashboard`
`/notifications`	`NotificationCenter`
`/chatbot`	`ChatbotInterface`

There is no catch-all route, so `NotFound.jsx` is never mounted, and the route guard logic is inside individual pages (each calls `authAPI.isAuthenticated()` and redirects).

`services/api.js`**.** One axios instance (`baseURL` = `VITE_API_URL` or `http://localhost:5163/api`, `withCredentials: true`). The response interceptor logs errors except for a list of "known unimplemented" cases and redirects to `/login` on any 401 unless already on login or register. Exported objects: `authAPI` (with the 30 s session cache), `portfolioAPI`, `accessRequestAPI`, `notificationAPI`, `projectAPI`, `experienceAPI`, `educationAPI`, `skillAPI`, `socialMediaLinkAPI`, `adminAPI`, `chatbotAPI`, `imageUploadAPI`, `statisticsAPI`.

Key pages.

- `Dashboard.jsx` (532 lines): lists visible portfolios, search and filters, and sends access requests.
- `PublicPortfolios.jsx`: public gallery with the same request-access action.
- `PortfolioView.jsx` (682 lines): renders a portfolio and exports it client-side with `html2pdf.js`; reads `localStorage.token`.

- `PortfolioEdit.jsx` (1946 lines, the largest file): tabbed editor for portfolio details and all five sections, calling the per-section APIs.
- `PortfolioManager.jsx`: owner's management view; image upload and server export download.
- `PortfolioCreate.jsx` and `PortfolioCreateModern.jsx` (page and component variants): creation flow.
- `AccessRequests.jsx`: inbox and outbox with approve and reject actions.
- `NotificationCenter.jsx`: lists and marks notifications read.
- `Profile.jsx`: reads the session user and updates profile or password.
- `AdminDashboard.jsx`: a Vanta-backed page with stat cards and no API calls in its first 40 lines (it appears to display static content).
- `ChatbotInterface.jsx` and `components/Chatbot.jsx`: full-page and floating chat widget with menu options and redirect actions.
- `Navigation.jsx`: top bar; checks session, shows user, logs out and clears several `localStorage` keys.
- Empty or duplicate files: `AnimatedBubbles.jsx` and `AdminDashboard_new.jsx` are zero bytes; `Dashboard.jsx` and `DashboardNew.jsx`, and `PortfolioView.jsx` in `pages/` and `components/`, are parallel variants.

6. Data models, configuration and environment

6.1 Entities

Entity	Key fields	Relationships
`User`	Id, FullName (100), Email (100), PasswordHash (JsonIgnore), Role (20, default "User")	has many Portfolios
`Portfolio`	Id, Title (100), Description, ProfileImage, IsPublic, five `Is*Public` flags, Email, Phone, City, Country, CreatedAt, UpdatedAt, UserId	belongs to User; has many children
`Project`	Title (100), Description, ImageUrl, ProjectUrl, timestamps	belongs to Portfolio
`Education`	Institution, Degree, Field, StartDate, EndDate?, Description	belongs to Portfolio
`Experience`	Company, Position, Location, StartDate, EndDate?, Description	belongs to Portfolio
`Skill`	Name (50), Level (1 to 5, default 3)	belongs to Portfolio
`SocialMediaLink`	Platform (50), Url, IconName (50)	belongs to Portfolio
`AccessRequest`	PortfolioId, RequesterUserId, Status (Pending, Approved, Rejected), Message (500), OwnerResponseNote (500), CreatedAt, DecidedAt?	Portfolio and Requester, cascade
`Notification`	UserId, Type (AccessRequested, AccessApproved, AccessRejected, PortfolioViewed), Title (200), Message (1000), IsRead, PortfolioId?, AccessRequestId?	belongs to User
`PortfolioViewLog`	PortfolioId, ViewerUserId?, ViewerName, ViewerEmail, ViewedAt, IpAddress, UserAgent	Portfolio cascade, Viewer set null

`PortfolioCreationState` and its nested `*Data` classes are plain classes describing a multi-step creation wizard; they are not DbSets and no code read here persists them. No unique index on `User.Email` is configured in `AppDbContext` or the models (uniqueness is checked in code only). The migrations folder has `InitialCreate` (2025-09-12) and `AccessAndNotificationSystem` (2025-09-17); `PortfolioViewLogs` is in the DbContext, so it should be confirmed that it is covered by the second migration or the model snapshot before deploying.

6.2 Configuration keys (`appsettings.json`)

Key	Purpose
`ConnectionStrings:DefaultConnection`	MySQL connection string (placeholder password)
`JwtSettings:SecretKey`	Present in config but not read by any code
`Admin:Pin`	Secret for `/Auth/promote` and admin password reset
`Admin:Contact:{Name,Email,Phone}`	Shown and emailed by forgot-password

Key	Purpose
`Smtp:{Host,Port,EnableSsl,Username,Password,FromEmail,FromName}`	SMTP settings
`OpenAI:ApiKey`	OpenAI key
`AllowedOrigins`	Optional string array for CORS (not in the template)
`Logging`, `AllowedHosts`	Standard (`AllowedHosts` is `*`)

Both committed files contain only placeholders. `appsettings.Development.json`, `appsettings.Local.json` and `appsettings.Personal.json` are git-ignored. ASP.NET Core also maps environment variables with `__` separators (for example `Smtp__Password`), which is what `DEPLOYMENT_GUIDE.md` and `CONFIG_SETUP.md` direct users toward.

6.3 Frontend configuration

- `VITE_API_URL`: API base URL, default `http://localhost:5163/api`.
- `vite.config.js` only enables the React plugin; there is no dev proxy.
- Tailwind config scans `index.html` and `src/**`; it defines custom keyframes (float-up, shimmer, gradient-wave and others).
- The root `package.json` lists Tailwind 4 dev dependencies while `Frontend/package.json` uses Tailwind 3.4.

7. Setup, run and deployment

7.1 Prerequisites

.NET 9 SDK, MySQL server, Node.js 18 or newer with npm. The EF CLI (`dotnet tool install --global dotnet-ef`) is needed to apply migrations.

7.2 Backend

1. Create a MySQL database (the template uses `dpw_db`).
2. Put real values in a git-ignored `appsettings.Development.json` (or use environment variables) for the connection string, `Admin:Pin`, SMTP and OpenAI keys.
3. Apply migrations and run:

```
dotnet restore
dotnet ef database update
dotnet run --launch-profile http
```

The API listens on `http://localhost:5163` (HTTPS profile: `https://localhost:7064`). Swagger UI is available in the Development environment.

7.3 Frontend

```
cd Frontend
npm install
npm run dev        # Vite on http://localhost:5173
npm run build      # production bundle in Frontend/dist
```

Set `VITE_API_URL` for non-local APIs. Uploaded images are returned as relative `/uploads/...` paths, so the frontend must prefix the API host when displaying them (the export generator hard-codes `http://localhost:5163`).

7.4 Helper scripts

`quick-start.bat`, `start-application.bat`, `start-backend.bat`, `start-frontend.bat` launch both processes on Windows; `setup.ps1` checks .NET, Node and npm and automates first-time setup. The messages in `start-application.bat` mention ports 7000 and 5000, which do not match `launchSettings.json`.

7.5 CI/CD workflows

File	Trigger	What it does
`ci-cd.yml`	push and PR to main	.NET restore, build, test; then Node 18 `npm ci`, a build step whose `run` only does `cd Frontend`, and deploy of `Frontend/dist` to GitHub Pages
`deploy.yml` and `github-pages.yml`	push and PR to main	Node 18 install and build, publish `Frontend/dist` to GitHub Pages
`main.yml`	push and PR to main	Not fully reviewed; four workflows overlap in purpose

Only the frontend is deployed by CI; no workflow deploys the API or database. `DEPLOYMENT_GUIDE.md` (311 lines) contains the manual hosting instructions.

8. Security and operational observations

Positive points: passwords are hashed with BCrypt; the auth cookie is `HttpOnly`; configuration committed to git contains placeholders only; admin controller is role-protected; ownership checks exist in create, update and delete paths of the child controllers; the register and login code does not reveal whether an email exists beyond the "Email already in use" message at registration.

Observed risks and defects:

1. **Cookie security.** `SecurePolicy = None` and `SameSite = Lax` are applied unconditionally, including production. Over HTTPS the cookie should be `Always` secure, and a cross-site frontend (for example GitHub Pages calling a separate API host) would need `SameSite=None; Secure`, otherwise cookies will not be sent.
2. **No CSRF protection.** Cookie auth with state-changing JSON endpoints and no antiforgery token; `Lax` offers partial mitigation only.
3. **Missing endpoints referenced by the frontend.** The SPA calls `AccessRequest/{id}/approve`, `AccessRequest/{id}/reject`, `AccessRequest/my/received`, `AccessRequest/my/sent`, `Admin/users/{id}` (DELETE), `Admin/users/{id}/role`, `Admin/stats`, `Admin/portfolio-stats`, `Admin/user-activity`, and `Chatbot/generate-portfolio`. None exist in the controllers read. As a result a private-portfolio request can be created but never approved through the API, which defeats the access workflow. The response interceptor and console overrides in `main.jsx` hide the resulting 404 and 500 errors rather than surfacing them.
4. **Login contract mismatch.** `Login.jsx` stores `response.data.token` and `response.data.userId` in `localStorage`, but the API returns only `{ message }`, so both values are `undefined` strings. `PortfolioView.jsx` reads `localStorage.token`. The backend also prints diagnostics about a JWT `Authorization` header although JWT is not configured.
5. **Admin PIN endpoint.** `POST /Auth/promote` is public and protected only by a static PIN in a header, with no rate limiting or lockout; anyone with the PIN can make any account an admin. The reset-password endpoint returns the new password in the HTTP response.
6. **Debug endpoints in production code.** `GET /AccessRequest/debug/{id}` is anonymous and returns the owner's name and the owner's recent notifications (title, message, read state). `Notification/debug` and `test-db` dump claims and stack traces. Several handlers return `ex.Message` and `ex.StackTrace` to the client.
7. **Information exposure.** `GET /Statistics/dashboard` is anonymous and returns names and emails of the five most recent users. `GET /Education` and `GET /Experience` (no id) return every row of every portfolio, including private ones. Listing endpoints return all portfolios, including private ones' basic metadata and contact fields (email, phone, city); section flags gate only the child lists.
8. **Unrestricted export.** `GET /Portfolio/{id}/pdf` has no authentication or privacy check, so private portfolios can be exported by id.
9. **Stored HTML injection.** `ProfessionalPDFGenerator` concatenates user-supplied title, description and other fields into HTML without any encoding (no `HtmlEncode` calls), and email templates were not seen to encode names either. A malicious portfolio can inject script into the exported document.

10 **Upload handling.** Only the file extension is validated; there is no explicit size limit (the framework default of about 28 MB request body applies), no content-type check, and the uploader is not recorded. ImageSharp decoding does provide a real image check since non-images throw. Files are re-encoded as JPEG but keep the original extension (a `.png` upload becomes JPEG bytes named `.png`). Static file responses reflect any `Origin` with credentials allowed, which weakens the CORS policy for everything under `wwwroot`.

11 **CORS and hosts.** `AllowedHosts: *`. CORS defaults are localhost only and must be configured for deployment.

12 **Email address change.** `PUT /Auth/profile` updates the email without a uniqueness check and without re-authentication; it does not refresh the cookie claims, which keep the old email until the next login.

13 **Fire-and-forget tasks.** `Task.Run` is used for emails, and `AccessRequestController` awaits SMTP inline, so an SMTP failure after the database commit yields a 500 even though the request was saved.

14 **Logging.** The code relies heavily on `Console.WriteLine`, including registration emails and per-request identity lines; there is no structured logging, audit trail or log redaction.

15 **OpenAI usage.** The prompt receives arbitrary JSON from the client (`[FromBody] object`), the key is attached as a default header on a typed client, and the user input is not length-limited. Any signed-in user can generate paid API calls; there is no rate limit or quota. User prompt inputs are interpolated into prompts without sanitisation (prompt injection is possible but only affects the caller's own reply).

16 **Build gap.** `ImageUploadController` uses `SixLabors.ImageSharp`, but the project file lists no ImageSharp package, so a clean `dotnet restore && dotnet build` is expected to fail unless the dependency arrives some other way. This should be verified. Unused heavy packages (`IronPdf`, `DinkToPdf`, JwtBearer) increase attack surface and build size.

17 **Test coverage.** The CI runs `dotnet test` but the repository contains no test project.

18 **Migrations.** The README suggests `Session` cookie settings that the code does not use; the real cookie settings are hard-coded in `Program.cs`.

9. Known limitations and suggested improvements

Functional gaps:

- Implement the missing access-request endpoints (approve, reject, received, sent) and write `DecidedAt`, `OwnerResponseNote`, the requester notification and `SendAccessDecisionAsync`, which already exists in `EmailService` but is not wired in the controllers read.
- Implement or remove the unused frontend admin and chatbot calls; make `AdminDashboard` call real data.
- Fix the login flow by deleting the `token` and `userId localStorage` usage or by returning a user payload from `/Auth/login`.
- Either generate real PDFs (the `/pdf` route returns HTML; `IronPdf` and `DinkToPdf` are referenced but unused) or rename the route and file name.
- `ChatbotController.GetHistory` is a stub, and `PortfolioCreationState` is unused.
- Add a catch-all `*` route so `NotFound` is shown, and add route guards for authenticated and admin pages.

Security hardening:

- Make cookie flags environment-aware (`Secure` always in production, `SameSite=None` where cross-site is required) and add antiforgery protection.
- Remove or restrict debug endpoints; stop returning exception text and stack traces; fix `[AllowAnonymous]` on the debug route.
- Require authentication and privacy checks on export; HTML-encode all user data in generated HTML and emails.

- Replace the static admin PIN with role management by existing admins, add rate limiting (`AddRateLimiter`) on login, register, promote and chatbot endpoints, and never return passwords in responses.
- Add a unique index on `Users.Email`, validate with DTO annotations (`[EmailAddress]`, `[StringLength]`, password strength), and enforce size limits on uploads.
- Move secrets to environment variables or a secret store; delete the unused `JwtSettings` key.

Engineering quality:

- Move query and mapping logic out of controllers (a single `PortfolioDto` mapper replaces the repeated copy-pasted projection blocks, which dominate `PortfolioController`), and add pagination to list endpoints, which currently load everything with multiple `Include` calls.
- Add a test project (unit tests for access rules, integration tests with a test database) and a real CI build for the API; consolidate the four overlapping workflows and fix the empty frontend build step in `ci-cd.yml`.
- Remove dead or duplicate code (`DashboardNew`, `AdminDashboard_new`, `AnimatedBubbles`, duplicated `PortfolioView` and `PortfolioCreateModern`), the unused NuGet packages and the root `package.json` Tailwind 4 dependency; split the 1946-line `PortfolioEdit` into smaller components.
- Replace console logging and the console-silencing overrides in `main.jsx` and `index.html` with structured logging (`ILogger`) and a proper error boundary, and remove the `vanta@latest` CDN reference in favour of a pinned version.
- Add a frontend dev proxy or a central helper for building image URLs instead of hard-coded `http://localhost:5163`.
- Update the README to reflect actual endpoints (`check-session`, not `check`; the README's `/api/upload/*` paths are really `/api/ImageUpload/*`; the access request and chatbot features) and the true configuration keys.